LILA BLACK

Player Journey Visualizer

Architecture Document

> **Design principle**
> A level designer should open a URL and see useful data immediately — no setup, no configuration, no engineer needed. Every technical decision in this document traces back to that.

Tech Stack

Layer	Technology	Reason
Data pipeline	Python · pandas · pyarrow	Parquet is a columnar analytics format not readable by browsers. One script converts all files to JSON offline, moving cost to deploy time — not user time.
Frontend	Vanilla HTML + JavaScript	No build step. No framework. No node_modules. The tool opens by dragging a file into a browser and deploys as a folder of static files.
Rendering	HTML5 Canvas API	89,000 events as SVG DOM nodes would freeze the browser. Canvas draws pixels directly and handles the full dataset in under 100ms.
Heatmaps	heatmap.js v2.0.5 (CDN)	Loaded from cdnjs.cloudflare.com at runtime. Not bundled in the repo. Requires internet for heatmap overlay. Zero config, identical output to building manually.
Hosting	Vercel (static)	Connected directly to the GitHub repo. Every push auto-deploys. No server runtime, no downtime, global CDN.

Data Flow

The pipeline runs once locally. The browser only ever fetches pre-processed JSON.

```
Raw .nakama-0 parquet files  (one file = one player in one match)
        |
        v
scripts/process_data.py
  1. Glob all .nakama-0 files recursively across all date folders
  2. Skip 0-byte files (one exists — pyarrow crashes on empty files)
```

```
  3. Read with pyarrow, decode event bytes  ->  UTF-8 string
  4. Detect bots: regex /^\d+$/ on user_id  (numeric = bot, UUID = human)
  5. Normalize timestamps per match: ts_rel = ts - min(ts)
  6. Apply coordinate formula  ->  store u, v as normalized 0-1 floats
  7. Output per-map JSON + match index JSON
        |
        v
public/data/
  matches.json                796 matches · metadata only · ~5KB
  events_AmbroseValley.json   61,013 events · ~297KB
  events_GrandRift.json        6,853 events  · ~137KB
  events_Lockdown.json        21,238 events  · ~46KB
        |
     Vercel CDN
        |
        v
Browser (index.html — single file, no build)
  - Fetches matches.json on startup  ->  populates all dropdowns
  - Lazy-fetches map JSON only when that map is selected
  - All filtering runs in memory  (no re-fetch on filter change)
  - Canvas re-renders on every interaction: filter, scrub, resize
  - heatmap.js loaded from cdnjs for density overlay layer
```

Coordinate Mapping

This is the most technically precise part of the implementation. The README provides exact configuration per map:

Map	Scale	Origin X	Origin Z
AmbroseValley	900	−370	−473
GrandRift	581	−290	−290
Lockdown	1000	−500	−500

```
# Python pipeline — computed once, stored in JSON
u = (x - origin_x) / scale          # normalized 0-1 horizontal
v = (z - origin_z) / scale          # normalized 0-1 vertical

# JavaScript frontend — applied at render time
pixel_x = u * canvas_width
pixel_y = (1 - v) * canvas_height   # y-flip: image origin = top-left
```

The y-flip on the last line is non-obvious but essential. Game world coordinates increase upward (standard 3D convention). Screen/image coordinates increase downward. Without inverting v, every event renders on the wrong half of the map.

By storing normalized u, v in the JSON (not pixel coordinates), the frontend renders correctly at any canvas size without recalculation. The formula is applied at render time by multiplying by the current canvas dimensions.

> **Validation result**
> Sampled 200 events per map and verified each computed pixel against the actual minimap image. 100% of events land on map terrain — zero in the black border region. The coordinate system is confirmed correct.

Note: The README states minimaps are 1024×1024px. Actual files are 2000×2000px (2000×1998 for GrandRift). Because the formula uses normalized UV coordinates, image dimensions have zero effect on accuracy — the result is the same proportion regardless of size.

Assumptions & Edge Cases

What I found	Assumption made	How I handled it
event column stored as bytes	b'Position' not 'Position'	Decoded with .decode('utf-8') in pipeline before any processing
Timestamps are match-relative ms	Not wall-clock time — ms since match start	Normalized per match: ts_rel = ts − min(ts). Sort by ts_rel for timeline
One 0-byte file in dataset	pyarrow crash on empty file	os.path.getsize() check before read — skip gracefully, continue
Only Feb 14 in provided data	5 days described in README, only 1 present	Pipeline handles any date range dynamically — other days load when present
Kill/Killed events missing	Human-vs-human combat absent from Feb 14	All 8 event types handled in code — render automatically with full dataset
Minimap images are 2000px not 1024px	README documents wrong size	UV normalization is size-independent — no impact on accuracy, documented

Major Trade-offs

Decision	What I chose	What I gave up
Static files vs. live API	Static JSON on Vercel — no runtime cost, instant loads	Must re-run pipeline manually when new match data arrives
Per-map files vs. one file	Per-map lazy loading — only fetch the map selected	Three separate files to manage; a fourth map means a new file
Canvas vs. SVG rendering	Canvas — handles 89k events without frame drops	No native DOM interactivity — tooltip hit-testing built manually
Vanilla JS vs. React	Vanilla — no build step, opens anywhere, zero setup	Less structured state management for complex interactions
CDN for heatmap.js	One script tag, zero config, no bundling needed	Heatmap overlay requires internet — won't render fully offline
Aggregate timeline playback	Max match duration as timeline range — shows density patterns	Events from different matches appear simultaneously, not a perfect replay

What I'd Do With More Time

Feature	Why it matters
Named zone overlays	Lockdown has labeled zones (Mine Pit, Gas Station, etc.) on the minimap. Clickable polygon boundaries with per-zone K/D/loot stats would let designers answer specific design questions directly on the map.
Day-over-day trend chart	A chart showing kill density or zone coverage shifting across Feb 10–14 would let designers spot whether a balance patch changed player behavior — the most common design question after a ship.
Bot divergence score	A single calculated metric: mean distance between bot and human movement centroids per map. Above threshold → bot sessions are automatically flagged as unreliable for design testing.
Bundle heatmap.js locally	Remove the CDN dependency. Tool would work fully offline and not rely on an external service being available.
Automated pipeline	GitHub Action running process_data.py on a schedule, committing updated JSON, triggering Vercel redeploy. Currently the pipeline requires a manual step from a developer.